Andres Huart
2026

# Code Review Summary Report

## I. Overview

This assignment focused on giving students a better grasp on the Code Reviewing process. It started by teaching us how to use the Pull Request function on GitHub, and then how to add a reviewer to your code. Then we were taught the review process, including the back and forth that can happen as another pull request may be asked to double-check the suggestions were implemented. My approach to this was to both familiarise myself with the industry standards for code review, as given to us in the assignment description, and to ensure that the feedback I gave was constructive and not simply tearing down someone's code and starting it from scratch, as that is not effective for anyone.

## II. Feedback Received and Given

My reviewer, Brianna Stewart, gave me a few main pieces of in-line feedback for my Boggle Solver implementation. The first involved making my solver more flexible to account for grids that are not squares. For another piece of feedback, my reviewer suggested that I should normalize the entire grid to be uppercase at the start of the code, rather than what I originally had which was uppercasing grid values during the recursive search. I was also given the feedback to remove my dictionary processing out of "getsolution()" as it is inefficient if the code solves the same dictionary multiple times. Beyond these larger ones, other pieces of smaller feedback included improving my naming consistency with PEP8, and replacing magic numbers in my code with constants, such as the minimum word length of "3". The main improvements, as they stated in their overall comment, were refinements to my code rather than a full overhaul.

When it was my turn to review, I wanted to make sure I got a grasp of Brianna Stewart's implementation of the Boggle Solver. After looking through it, I noted in the code how there was great usage of "prefix_set" as a means to prune the words, as well as a great implementation of backtracking. My feedback was, as follows, using a Prefix Tree to ensure the code does not take too much memory as the prefixes will be stored in a single path of nodes, rebuilding "prefix_set" to ensure users don't run into errors should the code encounter a word in a new dictionary that was not in the old one, adding an assumption following words that start with "Q" to handle the "QU" Boggle logic, and adding a check in "init" to ensure there is a rectangle grid to avoid an IndexError. I also noted that their solution to hardcode the minimum length as "3" does work, though should the user want to switch the length or change the game rules, it presents a challenge. I offered a suggestion to create a constant or variable that is set when the game starts, and the minimum length could be passed into the "init" method. I made sure to note the comments and notes of feedback were tweaks that could refine their implementation and that could strengthen the code even more, and that overall, Brianna's implementation uses great techniques such as pruning and backtracking to successfully create a Boggle Solver.

## III. Improvements Implemented

To implement the refinements to my Boggle Solver solution, I took Brianna's feedback into consideration. First, I made my solver more flexible by allowing it to accept rectangular grids instead of only square grids by tracking rows and columns independently to

allow grids of any dimension. I then fixed my implementation's efficiency. Originally, my "getSolution" method involved iterating through the entire dictionary and then stripping strings to build a prefix set. This is highly inefficient for sets of larger words. Thus, I improved it so that this process happens once with "set_dictionary", which makes repeated solves on the same dictionary faster. I also took Brianna's feedback of adding a magic number for the minimum word length of "3" and I added their feedback of normalization happening once at the input instead of during the recursion. All of these changes were, as Brianna mentioned in their end comment, refined the code, made it more flexible, and solved memory issues. These memory improvements on top of the flexibility makes the code more efficient, which is ideal for a solver that would be used for a word game such as Boggle.

**IV. Static Analysis Results**

The lint/style changes in my code that I fixed revolved around making sure that my Boggle Solver code matched the standards of PEP8. This involved using the snake_case naming convention for function throughout my code. I also added an underscore to the front of internal variables and the recursive helper function, as a single leading underscore signals that these attributes are for internal class use. I added constants for magic numbers at the top of a class, instead of having the minimum word length hardcoded in the middle of a loop. I fixed the spacing by adding spaces in between methods and logical blocks, and I broke long lines of code into multiple lines using parentheses and indentation. Lastly, the auto checker in Codio identified trailing whitespace issues that I did not notice, and so I removed them on the various lines identified by Codio, and then added a newline at the end of the file.

**V. Regression Testing**

However, with all of the changes made, it was important to test the code to make sure my original, though less refined, Boggle solver solution was not lost in the updates. This is where "Regression Testing" comes into play. In other words, testing the code after the changes to make sure the code's quality did not "regress" and that it both stuck to the assignment standards and maintained its effectiveness. To do this I relied on the Codio auto-checker found in this Starter Project assignment and past ones. Every time I added a new change to my code, from both Brianna's feedback and the lint/style changes in the Codio checker, I would run the code again through the Boggle Solver auto-checker to make sure the changes I implemented did not mess up the solution. Fortunately, every time I updated the code, the solution was not drastically changed, allowing it to pass the auto-checker and letting me know that the regression tests worked and my code was a successful Boggle Solver implementation even after the changes.

**VI. Reflection**

This assignment taught me that code reviews are just as important, if not at times more, than creating your implementation in the first place. They are a lengthy process that requires you to be attentive and constructive to ensure that you not only understand the feedback you receive, but also give proper feedback that allows the code of the person you're reviewing to improve. It also taught me that professional workflows may appear difficult to navigate at first, as the new GitHub features were in the beginning, but once you understand the features, they are a useful tool that structures and streamlines the review process. Overall, code reviews allow for not only your code to grow and improve, but for your skills as a software developer to grow as well.